

Relatório de Auditoria de Segurança

Ecossistema de Células — Missão Atos

Agosto de 2026

Escopo auditado: API NestJS (apps/api), painel web Next.js (apps/web), packages (domain, contracts, database, config) e artefatos de deploy (compose.yaml, GitHub Actions).

Nota metodológica: Stack detectada — TypeScript · Turborepo · NestJS 11 (Express) · Prisma 7 (PostgreSQL 18) · Next.js 16 App Router (React 19) · auth própria (JWT HS256 + refresh opaco HTTPOnly) · deploy: Render (API) + Vercel (web) via GitHub Actions; sem Dockerfile/Terraform/Helm. Cada categoria mapeada: (1) isolamento = RLS ausente → verificação do filtro por `churchId` via `principal.churchId` em toda query de listagem/consulta; (2) páginas client-side vs. gate de papel no servidor; (3) revisão sistemática de todos os handlers por ID; (4) segredos em código/config/histórico git; (5) XSS em sinks do framework (React).

1. Resumo Executivo

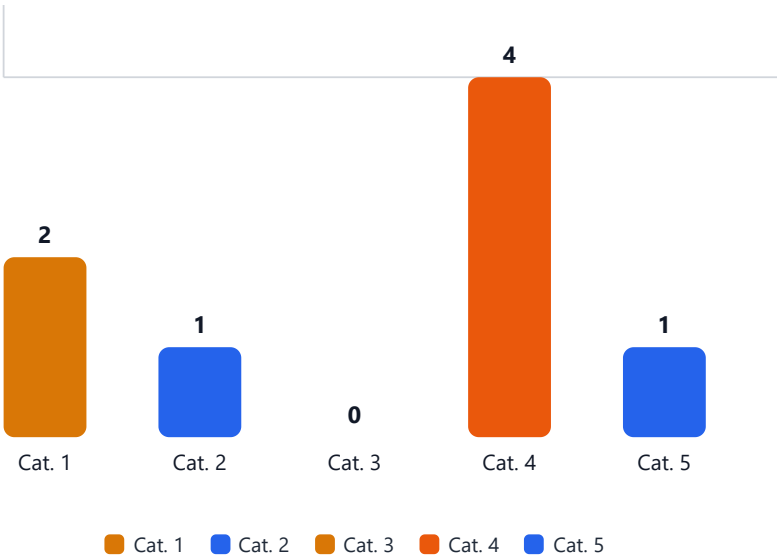
Foram identificados **8 achados** (6 acionáveis + informativos), sendo 1 de severidade alta. A base é sólida: isolamento por igreja aplicado em toda a camada de dados, autorização de servidor em toda rota sensível, ausência de sinks XSS no frontend e histórico git limpo. Os riscos centrais estão na **configuração de segredos e cookies** (tema 4) e na **ausência de RLS** no banco (tema 1).



Distribuição por Severidade



Distribuição por Categoria



2. Pontos Fortes e Pontos Fracos

2.1 Pontos fortes (verificados e corretos)

- Isolamento por igreja na camada de aplicação (Categoria 1 e 3).** Revisão sistemática de todos os 8 controllers (health, auth, users, people, church, cells, meetings, attendance) e de todos os repositories: toda query filtra por

`churchId` derivado de `principal.churchId`, e toda escrita usa chaves compostas que embutem `churchId` (`id_churchId`, `churchId_meetingId_personId`). Não foi encontrado IDOR cross-tenant.

- **Autorização de servidor em toda rota sensível (Categoria 2).** Todas as rotas de escrita e gestão carregam `@Roles(...)` + políticas de aplicação (`assertAdministrator`, `assertManager`, `ManageUserPolicy`, `ViewPersonPolicy`, `CellViewPolicy`, `MeetingEditPolicy`, `AttendancePolicy`). Nenhum gate de papel do frontend depende de verificação no navegador — a API valida o privilégio em toda rota sensível (ex.: `users.controller.ts:105,141,158,172,186,202,229,248`).
- **Tratamento de XSS no frontend (Categoria 5).** React escapa todo texto interpolado; não há `innerHTML`, `eval` ou `new Function`; não há href/src controlados por usuário (links internos de UUID + link WhatsApp estático); não há renderização de markdown/HTML vindo da API. O access token vive apenas em memória (não em `localStorage`/cookie), robustecendo contra XSS.
- **Higiene de segredos no git (Categoria 4).** `.env` está corretamente no `.gitignore` e nunca foi commitado. O histórico do git está limpo: nenhum segredo real de produção foi encontrado (apenas placeholders e valores locais `local_dev_only` / `replace-with-*`).
- **Criptografia da sessão e rate limiting.** Senhas com Argon2id; refresh token opaco com pepper e rotação de sessão (com detecção de reuso de token por família); cookie `httpOnly` + `sameSite strict`; rate limiting de login por IP e conta; CORS com allowlist de origens.

2.2 Pontos fracos (riscos centrais)

Os principais riscos são de **configuração e defesa em profundidade**, não de código funcional atravessado por tenant/IDOR:

- **Segredos padrão não rejeitados** (4.1, alta): placeholder de JWT/pépper passa na validação e vira segredo real se copiado para produção.
- **Cookie de refresh sem Secure permitido em produção** (4.2, média).
- **Sem RLS no PostgreSQL** (1.1, média): isolamento só na camada de aplicação em um modelo multi-inquilino.
- **Frontend sem autorização no servidor** (2.1, baixa): gates apenas client-side (defesa em profundidade).

3. Tabela de Achados

Severidade	ID	Arquivo:linha	Descrição
Alta	4.1	<code>packages/config/src/server.ts:12,17,31-36,69-77</code>	Validação de startup não rejeita segredos padrão públicos (<code>JWT_ACCESS_SECRET</code> / <code>REFRESH_TOKEN_PEPPER</code>)
Média	1.1	<code>packages/database/prisma/schema.prisma</code> · <code>apps/api/src/main.ts</code> (sem RLS)	Isolamento de inquilino feito apenas na camada de aplicação — sem Row-Level Security no PostgreSQL
Média	4.2	<code>packages/config/src/server.ts:24-27</code> · <code>.env:20</code> · <code>apps/api/src/modules/identity/presentation/cookie</code> · <code>s/refresh-cookie.service.ts:44-52</code>	<code>AUTH_COOKIE_SECURE</code> default false e nenhuma guarda de startup obriga Secure em produção
Baixa	2.1	<code>apps/web/src/shared/auth/guards.tsx:41-74</code> · <code>app/(authenticated)/layout.tsx</code> (sem <code>middleware.ts</code>)	Autorização do frontend é 100% client-side — sem proteção no servidor (Next.js middleware / Server Components)
Baixa	4.3	<code>apps/web/next.config.ts:8</code>	<code>API_PROXY_TARGET</code> aponta por padrão para endpoint de produção
Baixa	5.1	<code>apps/web/app/layout.tsx:32</code> · <code>apps/web/next.config.ts</code> (sem headers CSP)	<code>dangerouslySetInnerHTML</code> presente e nenhuma Content-Security-Policy configurada
Informativa	1.2	<code>apps/api/src/modules/cells/infrastructure/prisma-cells-management.repository.ts:287-291</code>	<code>findCellByCode</code> não filtra <code>deletedAt</code> (célula excluída logicamente pode bloquear o código)
Informativa	4.4	<code>.env:9,13,14,18</code> (arquivo local gitignored)	<code>.env</code> local contém segredos aparentemente reais (Neon DB, JWT secret, pepper) — histórico git limpo

4. Achados Detalhados por Categoria

4.1 Categoria 4 · Chaves expostas (hardcode)

Alta

Validação de startup não rejeita segredos padrão públicos (JWT_ACCESS_SECRET / REFRESH_TOKEN_PEPPER)

packages/config/src/server.ts:12,17,31-36,69-77

Descrição: `parseAuthenticationEnvironment` (server.ts:69-77) exige que os dois segredos tenham ≥ 32 caracteres e sejam distintos, mas **não rejeita os valores placeholder do** `.env.example`. Tanto `replace-with-at-least-32-random-characters` (37 chars) quanto `replace-with-a-different-32-character-secret` (46 chars) passam na validação. Um deploy que copie o `.env.example` para produção sobe a API com segredos de assinatura **publicamente conhecidos**.

Por que é explorável: Atacante pode assinar tokens JWT de acesso (claims `sub`, `churchId`, `roles[]` — inclusive `ADMIN`) e derivar refresh tokens usando a pepper conhecida, obtendo acesso total a todos os dados, células, pessoas e usuários da igreja.

Impacto: Comprometimento total da autenticação e autorização (account takeover de qualquer usuário).

Sugestão de correção: Rejeitar na validação de ambiente os valores placeholder conhecidos, exigir uma entropia mínima (ex.: `base64` ≥ 32 bytes aleatórios) e bloquear inicialização em produção se `NODE_ENV=production` com segredo padrão.

1.1 Categoria 1 · Banco sem tranca (isolamento de inquilino)

Média

Isolamento de inquilino feito apenas na camada de aplicação — sem Row-Level Security no PostgreSQL

packages/database/prisma/schema.prisma · apps/api/src/main.ts (sem RLS)

Descrição: O isolamento entre igrejas (tenant) é implementado exclusivamente filtrando `churchId` (derivado de `principal.churchId`) em cada repository Prisma. Não há **Row-Level Security (RLS)** no banco nem políticas por papel. O esquema é explicitamente multi-inquilino (`church_id` em toda tabela de negócio), mas a tranca do dado depende de cada query de aplicação lembrar de filtrar.

Por que é explorável: Um único caminho de consulta (listagem, busca, agregação, relatório futuro, exportação ou raw query) que omita o filtro `churchId` vaza dados de outra igreja. Hoje o deploy é single-tenant (`AUTH_CHURCH_ID` fixo), o que reduz a exposição prática, mas o modelo é multi-tenant e a defesa em profundidade no banco está ausente.

Impacto: Vazamento de dados entre igrejas se qualquer consulta esquecer o filtro; sem defesa em profundidade no banco.

Sugestão de correção: Configurar RLS no PostgreSQL com política por `church_id` e `session.church_id` setado por sessão autenticada, mantendo o filtro de aplicação como segunda camada.

4.2 Categoria 4 · Chaves expostas (hardcode)

Média

AUTH_COOKIE_SECURE default false e nenhuma guarda de startup obriga Secure em produção

packages/config/src/server.ts:24-27 · .env:20 · apps/api/src/modules/identity/presentation/cookies/refresh-cookie.service.ts:44-52

Descrição: `AUTH_COOKIE_SECURE` tem default `false` (server.ts:24-27) e o `.env` local define `false` (.env:20). O cookie de refresh é emitido sem a flag `secure` (refresh-cookie.service.ts:46-47) quando `false`. Não há validação de startup que force `Secure=true` quando `NODE_ENV=production`.

Por que é explorável: Em produção (Render/Vercel, HTTPS), se `AUTH_COOKIE_SECURE` não for sobrescrito para `true`, o cookie `httpOnly` de refresh viaja sem `Secure`. Qualquer downgrade a HTTP ou interceptação (ex.: rede corporativa, proxy, MITM em HTTP) permite captura do token de refresh e sequestro de sessão. A flag também muda o `path` do cookie (`/auth` vs `/api/auth`), acoplando a configuração e o roteamento.

Impacto: Sequestro de sessão por captura do cookie de refresh em conexões não criptografadas.

Sugestão de correção: Forçar `AUTH_COOKIE_SECURE=true` quando `NODE_ENV=production` (rejeitar `false` em produção) e manter um único `path` estável (`/api/auth`) independente do ambiente.

2.1 Categoria 2 · Permissão definida no navegador

Baixa

Autorização do frontend é 100% client-side — sem proteção no servidor (Next.js middleware / Server Components)

apps/web/src/shared/auth/guards.tsx:41-74 · app/(authenticated)/layout.tsx (sem middleware.ts)

Descrição: As checagens `RequireRole` (guards.tsx:41-61) e `Can` (guards.tsx:63-74) são componentes client (`"use client"`) e apenas **escondem a UI** com base em `principal.roles`. Não há `middleware.ts` nem roteamento protegido no servidor; as páginas são entregues ao navegador antes que o JS do cliente redirecione.

Por que é explorável: Não há escalonamento de privilégio real, pois a API **valida** todas as rotas sensíveis no servidor (ver Pontos Fortes). Porém é uma falha de defesa em profundidade: conteúdo/UI de admin é transferido a usuários sem permissão, e o sistema depende da correção da API como única barreira. Qualquer rota sensível futura adicionada sem `@Roles` /política seria exposta.

Impacto: Exposição de UI e marcação de papéis no cliente; depende de defesa única na API para segurança real.

Sugestão de correção: Adicionar `middleware.ts` /Server Components para autenticar e autorizar no servidor e não servir conteúdo restrito a não autorizados.

4.3 Categoria 4 · Chaves expostas (hardcode)

Baixa

API_PROXY_TARGET aponta por padrão para endpoint de produção

apps/web/next.config.ts:8

Descrição: `next.config.ts:8` usa `process.env.API_PROXY_TARGET ?? "https://eccosistema-celula.onrender.com"` — um endpoint real de produção como padrão. Um build/dev local sem a variável redireciona tráfego autenticado para a API de produção.

Por que é explorável: Sessões e requisições de um ambiente de desenvolvimento/palco são roteadas para a API de produção, gerando efeitos colaterais em dados reais ou exposição cruzada de ambiente. Não é um segredo, mas é config de produção embutida como default.

Impacto: Roteamento acidental de tráfego para produção e risco de operações sobre dados reais em ambientes de dev/teste.

Sugestão de correção: Remover o default de produção (obrigar a variável `API_PROXY_TARGET` por ambiente) ou torná-lo vazio/inválido por padrão.

5.1 Categoria 5 · Inputs sem tratamento (XSS)

Baixa

dangerouslySetInnerHTML presente e nenhuma Content-Security-Policy configurada

apps/web/app/layout.tsx:32 · apps/web/next.config.ts (sem headers CSP)

Descrição: app/layout.tsx:32 injeta via dangerouslySetInnerHTML o script de inicialização de tema. O conteúdo é uma constante estática (não controlada por usuário), então hoje não há injeção. Contudo, next.config.ts não define nenhum header de segurança — não há Content-Security-Policy (script-src, img-src), nem X-Frame-Options /HSTS.

Por que é explorável: Sem CSP, qualquer futura imagem data: ou sink de HTML não sanitizado (e o próprio dangerouslySetInnerHTML se um dia receber dado dinâmico) pode executar script. A ausência de CSP é o vetor habilitador para XSS no painel.

Impacto: Sem CSP, o risco de XSS futuro não é mitigado pelo navegador.

Sugestão de correção: Adicionar Headers de CSP estrita em next.config.ts (ex.: script-src 'self'; img-src 'self' data: https:; object-src 'none') e, idealmente, substituir o script inline por um arquivo estático.

1.2 Categoria 1 · Banco sem tranca (isolamento de inquilino)

Informativa

findCellByCode não filtra deletedAt (célula excluída logicamente pode bloquear o código)

apps/api/src/modules/cells/infrastructure/prisma-cells-management.repository.ts:287-291

Descrição: findCellByCode filtra por code e churchId, mas não por deletedAt. Já as demais queries de listagem filtram deletedAt: null. Código scoped por igreja (sem vazamento cross-tenant), porém uma célula soft-deleted pode continuar 'ocupando' o código na checagem de unicidade.

Por que é explorável: Sem impacto multi-inquilino; trata-se de inconsistência lógica intra-igreja que pode impedir reutilização de código de célula após exclusão lógica.

Impacto: Bloqueio acidental de código de célula; baixíssimo risco de segurança.

Sugestão de correção: Adicionar deletedAt: null ao where de findCellByCode para consistência com as demais consultas.

4.4 Categoria 4 · Chaves expostas (hardcode)

Informativa

.env local contém segredos aparentemente reais (Neon DB, JWT secret, pepper) — histórico git limpo

.env:9,13,14,18 (arquivo local gitignored)

Descrição: O .env do workspace contém valores de alta entropia que parecem credenciais reais de produção (DATABASE_URL da Neon com senha, JWT_ACCESS_SECRET, REFRESH_TOKEN_PEPPER, AUTH_CHURCH_ID). O arquivo é corretamente gitignored e não está no histórico do git (verificado: histórico limpo), então não vazou pelo repositório.

Por que é explorável: Mesmo sem commit, segredos reais de produção armazenados em .env local podem vazar por backup, máquina compartilhada, screenshot, logs ou compartilhamento acidental do arquivo. Recomenda-se confirmar se não são credenciais de produção e rotacioná-las.

Impacto: Caso sejam credenciais de produção reais, vazamento local comprometeria o banco e os tokens.

Sugestão de correção: Verificar/rotacionar as credenciais, nunca commitá-las, e usar cofre de segredos (secret manager da plataforma de deploy) em vez de .env local em produção.

5. Recomendações Priorizadas

Prioridade	Severidade	Ação
P1	Alta	Rejeitar segredos padrão/placeholder na validação de ambiente e exigir entropia mínima para JWT_ACCESS_SECRET e REFRESH_TOKEN_PEPPER; bloquear inicialização em produção com segredo público (findings 4.1).
P1	Média	Forçar AUTH_COOKIE_SECURE=true em produção (NODE_ENV=production) e unificar o path do cookie de refresh (finding 4.2).
P2	Média	Implementar Row-Level Security no PostgreSQL com políticas por church_id, mantendo o filtro de aplicação como segunda camada (finding 1.1).
P2	Baixa	Adicionar proteção de autorização no servidor do Next.js (middleware.ts / Server Components) para não servir UI restrita a não autorizados (finding 2.1).
P3	Baixa	Remover o default de endpoint de produção em API_PROXY_TARGET e adicionar headers de CSP estrita em next.config.ts (findings 4.3, 5.1).
P3	Informativa	Consistir findCellByCode com deletedAt e verificar/rotacionar eventuais credenciais reais no .env local (findings 1.2, 4.4).

6. Issues para o GitHub

Issues prontas em Markdown, abaixo do bloco `--- ISSUE n --- / --- FIM ISSUE n ---`.

ISSUE 1

[Segurança] Validação de startup aceita segredos JWT/pépper placeholder de conhecimento público

Labels sugeridas: `security, alta`

```
## Problema
A validação de ambiente da API (`parseAuthenticationEnvironment`) exige que `JWT_ACCESS_SECRET` e `REFRESH_TOKEN_PEPPER` tenham ≥32 caracteres e sejam distintos, mas **não** rejeita os valores placeholder do**`.env.example`**. Ambos os exemplares públicos (`replace-with-at-least-32-random-characters`, `replace-with-a-different-32-character-secret`) passam na validação por terem ≥32 chars.

## Por que é explorável
Um deploy que copie o `.env.example` para produção sobe com segredos de assinatura publicamente conhecidos. O atacante forja um JWT de acesso com `roles: ["ADMIN"]` (claims `sub`, `churchId`, `sid`) e deriva refresh tokens usando a pépper conhecida → acesso total à conta de qualquer usuário e a todos os dados da organização.

## Evidência
`packages/config/src/server.ts:12,17,31-36,69-77`
```ts
JWT_ACCESS_SECRET: z.string().min(32).optional(),
REFRESH_TOKEN_PEPPER: z.string().min(32).optional(),
...
const authenticationEnvironmentSchema = serverEnvironmentSchema.required({
 JWT_ACCESS_SECRET: true, REFRESH_TOKEN_PEPPER: true
});
// valida apenas: (1) presentes, (2) ≥32 chars, (3) distintos
if (parsed.JWT_ACCESS_SECRET === parsed.REFRESH_TOKEN_PEPPER) throw ...
```
`.env.example:16,22`

## Impacto
Account takeover total e comprometimento da autenticação/autorização de toda a plataforma.

## Sugestão de correção
Rejeitar um bloco de placeholders conhecidos; exigir entropia mínima (ex.: base64url ≥ 32 bytes); bloquear inicialização com `NODE_ENV=production` se o segredo for padrão.

## Critérios de aceite
- [ ] `parseAuthenticationEnvironment` rejeita os valores placeholder de `.env.example`.
- [ ] Rejeição cobre variações (`replace-with-*`, `change-me`, senha padrão etc.).
- [ ] Testes unitários cobrem os casos placeholder e a ausência dos segredos em produção.
- [ ] `.env.example` documenta a rotação obrigatória.
```

--- FIM ISSUE 1 ---

ISSUE 2

[Segurança] Cookie de refresh sem flag Secure em produção (AUTH_COOKIE_SECURE default false)

Labels sugeridas: security, média

```
## Problema
`AUTH_COOKIE_SECURE` tem default `false` e não há guarda de startup que o force a `true` quando
`NODE_ENV=production`. O cookie httpOnly de refresh é emitido sem `Secure`.

## Evidência
`packages/config/src/server.ts:24-27` (`AUTH_COOKIE_SECURE: z.enum(["true","false"]).default("false")`)
`apps/api/src/modules/identity/presentation/cookies/refresh-cookie.service.ts:44-52` (cookies sem `secure` quando
false; path muda entre `/auth` e `/api/auth`)
`.env:20` (`AUTH_COOKIE_SECURE=false`)

## Impacto
Sequestro de sessão por captura do token de refresh em conexões HTTP (downgrade/MITM) em produção.

## Sugestão de correção
Em `NODE_ENV=production`, exigir `AUTH_COOKIE_SECURE=true` (erro de startup se `false`) e fixar um único `path`
(`/api/auth`).

## Critérios de aceite
- [ ] Startup falha em produção com `AUTH_COOKIE_SECURE=false`.
- [ ] Cookie emitido com flag `Secure` em produção.
- [ ] `path` do cookie estável entre ambientes.
- [ ] Testes cobrem os cenários dev/prod.
```

--- FIM ISSUE 2 ---

ISSUE 3

[Segurança] Isolamento de inquilino sem Row-Level Security no PostgreSQL

Labels sugeridas: security, média

```
## Problema
O isolamento entre igrejas depende apenas do filtro `churchId` na camada de aplicação. Não há RLS no banco, que é
explicitamente multi-inquilino (`church_id` em toda tabela de negócio).

## Evidência
`packages/database/prisma/schema.prisma` (sem políticas RLS); todos os repositories filtram manualmente `churchId =
principal.churchId` (ex.: `prisma-cells-management.repository.ts:287-291`).
`apps/api/src/main.ts` – sem inicialização de RLS.

## Impacto
Uma única consulta (listagem/agregação/relatório/raw query) que esqueça o filtro vaza dados de outra igreja; sem
defesa em profundidade no banco.

## Sugestão de correção
Migração SQL para habilitar RLS com política por `church_id` usando `SET LOCAL app.church_id` por request
autenticado; manter filtro de aplicação.

## Critérios de aceite
- [ ] Migração RLS para tabelas de negócio com política por church_id.
- [ ] Sessão/request seta a variável `app.church_id` antes das queries.
- [ ] Teste de integração prova que usuário de igreja A não lê dados da igreja B mesmo via query crua.
- [ ] Filtro de aplicação preservado.
```

--- FIM ISSUE 3 ---

ISSUE 4**[Segurança] Autorização do frontend apenas client-side — sem middleware/Server Component no Next.js**

Labels sugeridas: security, baixa

```
## Problema
`RequireRole`/`Can` são client-side e só escondem a UI. Não há `middleware.ts` nem roteamento protegido no servidor.

## Evidência
`apps/web/src/shared/auth/guards.tsx:41-74` (componentes `"use client"`)
`apps/web/app/(authenticated)/layout.tsx` — sem checagem server; ausência de `apps/web/middleware.ts`.

## Impacto
Defesa em profundidade fraca: UI e conteúdo restrito são entregues ao navegador de não autorizados; a segurança real depende apenas da API.

## Sugestão de correção
Adicionar `middleware.ts`/Server Components para autenticar e autorizar no servidor e não servir conteúdo restrito.

## Critérios de aceite
- [ ] `middleware.ts` redireciona não autenticados para /login.
- [ ] Rotas de admin negadas no servidor para papéis sem permissão (HTTP 403/redirect).
- [ ] Testes E2E confirmam acesso negado no servidor.
```

--- FIM ISSUE 4 ---

ISSUE 5**[Segurança] API_PROXY_TARGET com default de produção + ausência de Content-Security-Policy**

Labels sugeridas: security, baixa

```
## Problema (agrupado)
(1) `next.config.ts` usa endpoint de produção como default de proxy; (2) nenhum header de CSP é enviado pelo painel.

## Evidência
`apps/web/next.config.ts:8` (`process.env.API_PROXY_TARGET ?? "https://eccosistema-celula.onrender.com"`) e
`apps/web/next.config.ts` (sem `headers`).
`apps/web/app/layout.tsx:32` (`dangerouslySetInnerHTML` para script de tema).

## Impacto
Roteamento acidental de tráfego de dev para produção; e, sem CSP, qualquer XSS futuro no painel não é mitigado pelo navegador.

## Sugestão de correção
Remover o default de produção do proxy; adicionar Headers de CSP estrita (`script-src 'self'; img-src 'self' data: https:; object-src 'none'`).

## Critérios de aceite
- [ ] `API_PROXY_TARGET` sem default de produção (ou vazio inválido).
- [ ] Headers CSP presentes nas respostas do painel.
- [ ] Navegação e logotipo continuam funcionando sob a CSP configurada.
```

--- FIM ISSUE 5 ---

Gerado automaticamente em docs/security-audit — pode ser regenerado com `npm run report:security` ou `node generate-report.mjs`.